Reflection and Traceability Report on Software Engineering

Team #16, The Chill Guys
Hamzah Rawasia
Sarim Zia
Aidan Froggatt
Swesan Pathmanathan
Burhanuddin Kharodawala

1 Changes in Response to Feedback

The subsections below document **capstone peer-review** feedback from GitHub issues. For each item we state the source, a concise summary of the issue, our response (what changed), and traceability links. The main pull request bundling MIS, VnV Plan, VnV Report, and related CI workflow updates is PR #336; its description lists `Closes` for issues #165, #167, #168, #169, #244, and #245 so those issues close when the PR merges to `main`.

Throughout the course of the capstone project, we have recieved feedback from TAs, the instructor, teammates, other teams, the project supervisor (if present), and from user testers. We have carefully considered and assessed all of the feedback and carefully integrated them all into our project

1.1 ProblemStatementAndGoals and DevelopmentPlan

The following items addresses **ProblemStatementAndGoals** and **DevelopmentPlan** feedback from **TA and peer reviewers** (GitHub issues). We accepted the feedback and updated `docs/ProblemStatementAndGoals/` (source `.tex` and rebuilt `ProblemStatementAndGoals.pdf`) and `docs/DevelopmentPlan/` (source `.tex` and rebuilt `DevelopmentPlan.pdf`); revision history in the Problem Statement and Goals documents the April 1st updates.

#258 — Overlapping goals, stakeholder description, and justification.
Source: TA and peer review (GitHub issue).
Summary of issue: The goals had some overlap and needed clarification. Moreover, the stakeholder description was thin and could be more specific to the campus context. Lastly, the justification needed to be more detailed for some goals. Details are summarized in the issue's description.
Response: We removed the overlapping goals, provided clear distinction

between them and justified them by providing specific measurability metrics. Details are summarized in the issue's description.
Traceability: Issue #258; changes in PR #333 (`docs/ProblemStatementAndGoals/ProblemStatementAndGoals.tex`).

#258 — Development plan schedule, CICD clarification. *Source:* TA and peer review (GitHub issue).
Summary of issue: The development plan schedule was unclear, and the CICD process needed more detail. Details are summarized in the issue's description.
Response: We clarified the development plan schedule and provided more detailed information about the CICD process. Details are summarized in the issue's description.
Traceability: Issue #258; changes in PR #333 (`docs/DevelopmentPlan/DevelopmentPlan.tex`).

1.2 SRS and Hazard Analysis

The following items address **SRS** and **Hazard Analysis** feedback from **TA, peer reviews and user feedback** (GitHub issues). We accepted the feedback and updated `docs/SRS/SRS_meyer` (source `.tex` and rebuilt `SRS_meyer.pdf`) and `docs/HazardAnalysis/HazardAnalysis.tex` (source `.tex` and rebuilt `HazardAnalysis.pdf`); revision history in the SRS and Hazard Analysis documents the April 1st update.

#254 — Overlapping requirements, missing sections, unclear goals and justifications.
Source: TA, peer reviews and user feedback (GitHub issue).
Summary of issue: The SRS documents had overlapping requirements, missing sections, and unclear goals and justifications. Details are summarized in the issue's description. Moreover, after usability testing, certain users suggested improvements in terms of adding new geolocation feature. This allows users to track their driver and navigate better.
Response: We addressed the issues in the SRS document by revising the overlapping requirements and clarifying the goals and justifications. Details are summarized in the issue's description. Users feedback was also incorporated by adding features as functional requirements in the document.
Traceability: Issue #254; changes in PR #334 (`docs/SRS/SRS_meyer.tex`).

#254 — Identifying more complex risks. *Source:* TA, peer reviews (GitHub issue).
Summary of issue: The hazard analysis document was missing some complex risks that could arise from the use of the app. Details are summarized in the issue's description. Moreover, after usability testing, certain users suggested improvements in terms of adding new geolocation feature. This allows users to track their driver and navigate better.

Response: We added those suggested risks to the hazard analysis document and provided mitigation strategies for them. Details are summarized in the issue's description. Users feedback was also incorporated by adding features as functional requirements in the SRS document.
Traceability: Issue #254; changes in PR #334 (`docs/HazardAnalysis/HazardAnalysis.tex`).

1.3 Design and Design Documentation

The following items address **Module Interface Specification (MIS)** and **Module Guide (MG)** feedback from peer reviewers and the TA Rubric. We accepted the feedback and updated `docs/Design/SoftDetailedDes` and `docs/Design/SoftArchitecture`.

#245 — Notation (§4) layout and type definitions. *Source:* Peer review (GitHub issue).
Summary of issue: The primitive-types table was narrow and hard to read; `UpdatedFields` used optional-field "?" style notation that did not match how partial updates are described elsewhere.
Response: We expanded the primitive-types table to full text width using `tabularx`; we rewrote `UpdatedFields` in prose so each field may be present or omitted independently, without placeholder syntax.
Traceability: Issue #245; PR #336 (`docs/Design/SoftDetailedDes/sections/04_notation.tex`).

#244 — Introduction link to the repository. *Source:* Peer review (GitHub issue).
Summary of issue: The MIS should point readers to the implementation repository; the PDF showed only "can be found at" because `\href` was missing its visible-text argument.
Response: We replaced the link with `\url{https://github.com/hitchly/hitchly}` so the URL is both visible and clickable in the PDF. We also fixed a typo (Hitchtly to Hitchly).
Traceability: Issue #244; PR #336 (`docs/Design/SoftDetailedDes/sections/03_introduction.tex`).

#348 — Vague Module Specifications and Exception Handling. *Source:* TA Rubric Evaluation (GitHub issue).
Summary of issue: The MIS lacked rigorous formal specifications. Exceptions were defined in vague terms, and some composite types were missing from the Notation section. The MG timeline lacked specific task assignments.
Response: We formalized the MIS by defining all state/environment variables and access program syntax. We rewrote every exception to trigger based on strict logical conditions. We added missing types to the Notation section and updated the Timeline to assign specific developer names instead of "All."

Traceability: Issue #348; PR #336 (`docs/Design/SoftDetailedDes/MIS.tex`, `docs/Design/SoftArchitecture/MG.tex`).

#243 — AC2 Traceability is Too Broad. *Source:* Peer review (GitHub issue).
Summary of issue: Anticipated Change 2 (User Interface) was mapped to almost every module in the system, reducing the clarity of information hiding.
Response: We restricted AC2 traceability in the MG to only the core UI-heavy modules (M2, M3, M4) to accurately reflect where UI changes would actually be isolated.
Traceability: Issue #243; PR #336 (`docs/Design/SoftArchitecture/MG.tex`).

#246 — Clarify Overlapping Module Boundaries. *Source:* Peer review (GitHub issue).
Summary of issue: The responsibilities of M7 (Rating), M8 (Safety), and M10 (Admin) seemed to overlap heavily regarding reports and analytics.
Response: We rewrote the "Secrets" and "Services" for these modules in the MIS to strictly separate their boundaries: user-to-user trust metrics (M7), raw incident data collection (M8), and platform-wide moderation/analytics (M10).
Traceability: Issue #246; PR #336 (`docs/Design/SoftDetailedDes/MIS.tex`).

1.4 VnV Plan and Report

The following items address **Verification and Validation Plan** and **VnV Report** feedback from peer reviewers and the TA Rubric. We accepted the feedback and updated `docs/VnVPlan`, `docs/VnVReport`, and rebuilt the corresponding PDFs.

#169 — Recording and evaluation of test results. *Source:* Peer review (GitHub issue).
Summary of issue: The plan did not clearly state where results are stored, how pass/fail is decided, or how defects are tracked.
Response: We added an unnumbered subsection under System Tests (§3) describing storage, pass/fail rules for automated, manual, and inspection-style tests, and defect handling.
Traceability: Issue #169; PR #336 (`docs/VnVPlan/VnVPlan.tex`).

#165 — Non-functional system tests (performance, security, scalability). *Source:* Peer review (GitHub issue).
Summary of issue: System-level testing for NFRs beyond reliability and usability was thin or missing relative to SRS-style expectations.
Response: In the VnV Plan we added §3.2.3–3.2.5 with concrete test cases. In the VnV Report we extended §4 with matching evaluation areas and

test cases.
Traceability: Issue #165; PR #336 (`docs/VnVPlan/VnVPlan.tex`, `docs/VnVReport/sections/04_nonfunctional_requirements.tex`).

#167 — Traceability in the VnV Plan. *Source:* Peer review (GitHub issue).
Summary of issue: The plan's requirements-to-tests mapping needed to cover profile/scheduling (FR-2), ratings/reporting (FR-5), and NFR-3–NFR-5 alongside existing rows.
Response: We expanded the §3.3 traceability table with those requirement IDs and pointers to the new NFR subsections.
Traceability: Issue #167; PR #336 (`docs/VnVPlan/VnVPlan.tex`).

#168 — VnV Report trace tables and NFR alignment. *Source:* Peer review (GitHub issue).
Summary of issue: Functional vs. non-functional captions and matrices did not fully align with SRS G.4 wording.
Response: We corrected §9 captions, added a `test-FR2-3` row, and replaced the NFR table with an NFR-1–NFR-5 matrix consistent with the updated plan.
Traceability: Issue #168; PR #336 (`docs/VnVReport/sections/09_trace_to_requirements.tex`).

Repository CI — (supporting the same PR). *Source:* Internal team (merge friction on documentation PRs).
Summary of issue: Required GitHub checks stayed on "Expected — Waiting for status" because path-filtered jobs were skipped and LaTeX runs cancelled incorrectly.
Response: We updated `.github/workflows/ci.yml` so named jobs always report success when paths do not match, and disabled `cancel-in-progress` for CI and `buildtex`.
Traceability: PR #336 (`.github/workflows/ci.yml`, `.github/workflows/buildtex.yml`).

#349 & #341 — Duplicate Objectives and Incomplete System Test Coverage. *Source:* TA Rubric Evaluation and Peer Review (GitHub issues).
Summary of issue: The VnV Plan objectives were duplicated. Tests did not cover all SRS requirements (e.g., Availability Updates, Location Security) and lacked explicit static testing procedures.
Response: We cleaned up Section 1.2, added test areas for FR-6 (Availability) and NFR-6 (Location Security), and detailed a static inspection code-review procedure for coordinate masking.
Traceability: Issues #349 and #341; PR #336 (`docs/VnVPlan/VnVPlan.tex`).

#341 — Redundant Team Member Roles. *Source:* Peer review (GitHub issue).
Summary of issue: Team member roles in the VnV Plan were highly

repetitive.
Response: We factored out common responsibilities into a shared "General Developer Responsibilities" block and specified unique testing focus areas for each member.
Traceability: Issue #341; PR #336 (`docs/VnVPlan/VnVPlan.tex`).

#342 — Test Execution Clarity. *Source:* Peer review (GitHub issue).
Summary of issue: The VnV Report did not specify whether functional tests were performed manually or via automated scripts.
Response: We added an explicit testing methodology statement to Section 3 of the VnV Report clarifying that functional requirements were validated manually via the Expo client.
Traceability: Issue #342; PR #336 (`docs/VnVReport/VnVReport.tex`).

#350 & #342 — Vague Nonfunctional Results. *Source:* TA Rubric Evaluation and Peer review (GitHub issues).
Summary of issue: Nonfunctional test results used vague terms like "most participants" or "same dataset," lacking compelling numeric evidence.
Response: We updated Reliability testing to explicitly state the use of randomized datasets. We updated Usability results with hard metrics (10 participants, 90% success rate, 102s median time, 72.6 SUS score) and linked to the Usability Report.
Traceability: Issues #350 and #342; PR #336 (`docs/VnVReport/VnVReport.tex`).

#342 — Lack of Documented Bugs Found. *Source:* Peer review (GitHub issues).
Summary of issue: The VnV Report showed "Pass" for all tests but failed to document any defects found or changes made due to testing.
Response: We added specific examples to Section 7 of the VnV Report, detailing a 0-seat capacity backend bug caught during unit testing, and UI/UX friction points discovered during manual usability testing.
Traceability: Issue #342; PR #336 (`docs/VnVReport/VnVReport.tex`).

#342 — CI/CD Failure Handling. *Source:* Peer review (GitHub issue).
Summary of issue: The report omitted what happens when a continuous integration test fails.
Response: We added a "CI/CD Failure Handling" section explaining that GitHub Actions strictly blocks pull requests from merging upon test failure.
Traceability: Issue #342; PR #336 (`docs/VnVReport/VnVReport.tex`).

2 Extras

The team completed two extras that were included in the scope of the project: a **usability testing report** for Hitchly and an **in-app first-use guide** so new

users see information about core features when they first open the app after sign-in.

2.1 Usability testing report

We produced a standalone usability testing document under `docs/UsabilityTesting/`. It describes goals, methodology, task scenarios for ten moderated sessions, moderator questions and surveys (including SUS), quantitative metrics (task success, time on task, assists), and results with recommendations. Task design aligns with the functional evaluation areas in the Verification and Validation report so usability findings can be read alongside formal requirement testing. This extra closes the loop between whether the implementation behaves correctly and whether real users can complete the main jobs without unnecessary friction.

2.2 In-app first-use guide (tutorial)

We added a lightweight, role-specific tutorial that appears when a user enters the main app as a **rider** or **driver** for the first time (until they finish or skip it). Content is delivered as a swipeable carousel of short slides covering welcome messaging and how to use primary navigation and key actions for that role. Completion is stored per user (rider vs. driver tutorial flags) via the backend so the guide does not repeat every launch. Implementation lives under `apps/mobile/features/onboarding/` (e.g. `views/RiderTutorial.tsx`, `views/DriverTutorial.tsx`, shared `components/Carousel.tsx`, and `hooks/useTutorialState.ts`), with the tutorial mounted from the rider and driver app layouts. Together with the written usability report, this in-product guidance supports learnability and reduces reliance on external documentation for first-time use.

3 Design Iteration (LO11 (PrototypeIterate))

3.1 Deployment-driven changes (verification email)

McMaster email verification is a central component of user trust and safety. We initially sent OTP mail over SMTP. After deploying the API to Render, SMTP proved unreliable in that environment (connection and deliverability issues typical of hosted platforms without a dedicated mail relay). We replaced direct SMTP with Brevo's REST API: `packages/emails/client.ts` posts to Brevo's transactional endpoint, and `apps/api/lib/email.ts` wires the client using `BREVO_API_KEY`. The reason for the design change is operational, since verification emails must send consistently in production without fighting the host's network constraints.

3.2 Admin experience and security posture

Early thinking placed admin capabilities inside the mobile app for privileged accounts. That approach risked shipping a hidden surface inside a consumer binary and limited what operators could see. The final design moves administration to a standalone web portal (`apps/admin`) backed by dedicated admin server modules (`apps/api/trpc/routers/admin/`), with dashboard areas for users, safety, operations, health, finances, and related views. The reason for the change is the richer operational insight (activity, logs, infrastructure-style summaries as implemented) and alignment with common practice, where administration is not a secret mode in the rider/driver app.

3.3 Live rides: location and Google Maps

Static match-time estimates were not enough once trips run in the real world. We expanded Google Maps–backed behaviour on the server for distance and ETAs during active trips. The `location` router adds `getTripDriverLiveLocation` (`apps/api/trpc/routers/location.ts`), returning the driver's live coordinates, distance/ETA toward pickup or drop-off depending on request status. The rider experience was updated under `apps/mobile/features/trips/` so live driver location, freshness, and external maps are visible during a ride. Mobile platform configuration (`apps/mobile/app.config.ts`) was adjusted for location permission copy and background behaviour so in-trip tracking matches store policies. Together, this addresses the client need for transparency while a ride is underway, not only at booking time.

3.4 Mobile configuration in TypeScript

We moved mobile app configuration from a JSON manifest to a typed module (`apps/mobile/app.config.ts`). That change was driven by iteration, as permissions, icons, and Google service wiring grew, a typed config reduced silent mistakes and made updates easier to review alongside code changes.

3.5 Iteration from usability findings

Formal usability testing is documented under `docs/UsabilityTesting/`. Sessions highlighted friction around campus drop-off choice, first-time to/from campus wording, cost visibility on match cards, and discovery of review and report entry points—see results and recommendation tables in that report. In response we carried a post-study interface pass to align visual patterns with familiar rideshare products where appropriate, tighten colour consistency, and improve labels and hierarchy on flows that had caused hesitation. We also shipped the in-app rider/driver tutorials summarized in Section 2 (Extras) of this document so first-time users get in-product guidance instead of inferring every primary action from a cold start. Those changes complement fixes tracked from the usability tables.

4 Design Decisions (LO12)

This section covers **constraints**, **assumptions**, and **limitations**, then summarizes how those factors justify the main architectural and product choices.

4.1 Constraints

Community and platform scope. Hitchly is scoped to the McMaster community with email-domain verification, and to modern iOS and Android clients. That constraint drove a mobile-first product with server-enforced affiliation checks rather than an open web signup.
Privacy and payments. Canadian privacy expectations (e.g. PIPEDA, as reflected in the SRS) and card-data rules (PCI DSS) pushed us toward minimal sensitive storage and payment flows that delegate card handling to Stripe instead of persisting raw payment credentials in our database.
Engineering standards. The SRS calls for TypeScript with linting and formatting enforced in CI. That constraint reinforced a typed codebase end to end and predictable review gates, which mattered for a multi-contributor capstone schedule.
Time, team size, and hosting. A fixed academic term and small team constrained how much custom infrastructure we could own. Deploying the API on a managed host (e.g. Render) with a managed PostgreSQL provider (e.g. Neon) traded operational control for reliability and speed of iteration. In practice, that hosting model also constrained outbound email, as direct SMTP was unreliable from the deployment environment, which motivated the move to a transactional email API as mentioned in the previous section.

4.2 Assumptions

User conduct and honesty. We assume users act in good faith during trips and that a McMaster-affiliated email (plus driver-license capture for drivers) is an acceptable affiliation signal for a campus pilot, not a substitute for commercial background screening.
Legal compliance by drivers. We assume drivers meet Ontario licensing, insurance, and vehicle rules as stated in the SRS narrative; the app prompts and records verification artifacts but does not replace government or insurer checks.
Technical environment. We assume participants run a current Expo build, stable network access, and that third-party services (maps, email, payments) remain available. We also assume campus and urban travel dominates, so Maps-backed distance, routing, and landmark-style drop-offs remain usable, and rural edge cases are weaker.
Operations. We assume few trusted operators use admin tooling, which supports a separate admin surface with role-appropriate access rather than embedding admin features in consumer apps.

These assumptions justified lighter onboarding than a commercial rideshare platform while still documenting trust and safety paths (reports, reviews, admin visibility).

4.3 Limitations

Verification depth. We do not run institutional background checks or insurance verification beyond license upload and email proof, and risk remains with users and institutional policy, as noted in the SRS limitations.
Dependence on third parties. Maps, email, hosting, and payment providers introduce cost, quota, and outage risk, and behaviour degrades if an API is throttled or unavailable.
Quality and coverage. Automated and manual testing are substantial but not exhaustive under course time pressure, and the primary UI language is English unless otherwise specified in the product.

4.4 Principal architectural and product choices

Given the constraints above, we adopted: tRPC with shared types across mobile, API, and admin to reduce contract drift in a small team; a pnpm monorepo with Turbo so shared packages (database access, email client, config) stay in one repository; PostgreSQL for relational integrity on users, trips, and matches; and a standalone admin web application separate from rider/driver clients for security, clearer roles, and richer operational views. These choices follow from the need for maintainability, type safety, and clear trust boundaries rather than from novelty for its own sake.

5 Economic Considerations (LO23)

Hitchly is software service mobile application aimed at a bounded campus market.

5.1 Market need

There is a plausible niche market among McMaster-affiliated commuters, which consists of riders who want lower-cost trips than solo driving, public transit, or occasional rideshare apps such as Uber or Lyft, and drivers who want to defray parking, fuel, and wear on recurring routes. A campus-scoped network can emphasize trust (shared institution) and repeat patterns (to/from campus) in ways that differ from general-purpose rideshare for ad-hoc city travel.

5.2 Marketing and adoption

Hitchly would be marketed through channels that already reach commuters: student unions and clubs, faculty and staff associations, sustainability and transportation offices, residence orientation, digital signage, and partnerships with

a few high-commute programs as a pilot cohort. The message combines cost sharing with verified McMaster affiliation (email-domain verification). User acquisition is organic and institutional through downloads, verified sign-ups, and completed trips measure traction.

5.3 Cost to build and run

One-time / sunk cost is primarily engineering time to reach a pilot-ready mobile client, API, payments, and operations tooling (capstone-scale effort). Ongoing operating cost scales with usage and includes managed API and database hosting (e.g. Render and Neon-style Postgres), transactional email (e.g. Brevo), Google Maps usage (distance, directions, and in-trip location workflows), Stripe processing and Connect-related fees, mobile store accounts, and domain/DNS. Exact monthly cost depends on trip volume, API call patterns, and chosen service tiers. The team would budget using provider dashboards once traffic is non-trivial.

5.4 Pricing and revenue model

Riders and drivers obtain a fare estimate derived from trip distance and time, and the implementation records a platform fee as a percentage of the charged amount. The deployed configuration applies a 15% platform fee, with the remainder intended for the driver after Stripe processing. Alternatives for a broader rollout include a university-paid deployment (no per-trip fee for users), a lower take rate for a pilot, or a subscription for verified power users, but the current model favours usage-based revenue tied to completed trips.

5.5 Illustrative break-even framing

Because Hitchly is not sold as discrete units, "how many units to make money" maps to how much trip volume (or institutional funding) covers monthly OPEX. Illustrative only suppose combined hosting, email, maps, and fixed third-party overhead were on the order of a few hundred Canadian dollars per month at pilot scale, and suppose an average completed trip produced roughly $1.50 in platform revenue after Stripe (15% of a modest $10 trip, ignoring Stripe's own fee in this sketch). Then on the order of 200–400 completed trips per month would approach that overhead band. This is not a forecast, only a check that revenue is volume-driven and sensitive to Maps and payment fees. A formal business plan would replace these round numbers with measured usage and provider invoices.

5.6 Potential user base

McMaster's public institutional reporting places total student headcount in the mid-30,000s in recent years, with additional faculty and staff. Not everyone

commutes by car or needs rideshare, so the addressable subset is smaller, commuters, occasional riders, and drivers with spare seats, but the ceiling for a McMaster-only product is still in the tens of thousands of affiliated people. Growth beyond that would require a different product scope (multi-campus or public launch), which is outside the current scope and focus of the project.

6 Reflection on Project Management (LO24)

6.1 How Does Your Project Management Compare to Your Development Plan

The Development Plan (`docs/DevelopmentPlan/DevelopmentPlan.tex`) committed to frequent meetings (including TA sessions on Wednesdays), a weekly team review, Discord as the primary real-time channel, GitHub for code and task tracking (Issues and Project boards), complementary use of Microsoft Teams and Outlook for files and formal contact, named functional leads (meeting chair, reviewer, testing, UI/UX, data) with cross-cutting development and documentation, and a Git workflow of feature branches, pull requests with reviewers, and merge to `main` after review.

In practice we largely followed that structure. We held regular standing team meetings on Discord, used GitHub Issues and pull requests for deliverables (including documentation feedback traced in Section 1 of this report), and relied on GitHub Actions for continuous integration (type-check, lint, format-check, build, tests, and LaTeX checks where applicable). Roles stayed flexible, leads provided direction, but every member contributed to development, tests, and docs as the plan intended. We used the technology stack described in the plan and README: a pnpm/Turbo monorepo, Expo (React Native) for the mobile app, tRPC for the API, PostgreSQL with Drizzle, Stripe and related services where needed, and GitHub for version control and CI—consistent with the plan's emphasis on TypeScript, testing, and automated checks before merge.

Deviations were mostly environmental or iterative rather than abandoning the plan. Hosting and email details evolved (e.g. production API deployment and transactional email via a REST provider, as noted in Section 3), and the admin experience moved to a separate web app for operational clarity. Those choices do not contradict the Development Plan's workflow or tooling principles, they refined where some services run while keeping the same collaboration and quality gates.

6.2 What Went Well?

- **Issue-driven traceability.** Feedback from TAs and peers was captured in GitHub issues and addressed in linked pull requests, which made accountability and rubric alignment easier than ad hoc chat alone.
- **Pull requests and CI.** Required checks before merging caught integration and style problems early and gave reviewers a single place to discuss

changes.

- **Discord + GitHub.** Discord worked well for quick coordination and meeting logistics, and GitHub remained the central source for tasks and code history.
- **Shared reviews.** Multiple teammates reviewed PRs, not only a single "reviewer" role, which improved quality and shared context.
- **Project board and milestones.** Using GitHub Projects helped group work by deadline and status when the team kept it updated.

6.3 What Went Wrong?

- **Deadline clustering.** We sometimes concentrated work too close to deliverable deadlines, which increased stress and reduced buffer for unexpected integration or CI issues.
- **Issue hygiene.** We could have created and triaged issues earlier and more consistently so the repository and board always reflected upcoming work at a glance.
- **Inconsistent meeting agendas.** During heavy documentation or demo weeks, meeting agendas occasionally were reduced to quick status-only updates with less time for meaningful discussion than the Development Plan's ideal structure.
- **Process vs. course load.** Competing coursework sometimes delayed reviews or board updates despite the team's intentions.

6.4 What Would you Do Differently Next Time?

- **Earlier breakdown of milestones.** Decompose each course deadline into issues two weeks ahead, with explicit "integration" tasks before the final merge.
- **Smaller, more frequent PRs.** Prefer narrow scopes and shorter-lived branches to reduce review load and merge conflicts near the end of a deliverable.
- **Definition of Done.** Agree per issue on reviewer, tests, and doc updates before marking work complete, so nothing ships with implicit assumptions.
- **Backup owners.** Name a secondary assignee for critical path items to reduce single points of failure during high stress periods.
- **Standing integration slot.** Reserve a set amount of time dedicated to merging and fixing CI on `main`, not only feature work.

7 Ethics and Equity (LO18)

Hitchly sits between several stakeholder groups: riders and drivers who share cost and risk on real trips, admins who must respond to safety and quality issues, the McMaster students and staff we intend as users for this campus-focused pilot, and the wider campus-area community affected by congestion and emissions. The app uses McMaster email verification and is meant for the McMaster community by design, it remains a student capstone project, and McMaster University does not operate or oversee the platform. Ethical design still required us to treat those groups fairly under the same rules, collect only proportionate data, and give people clear paths to trust and recourse, not optimizing for one role at the hidden expense of another.

We enforced a common access rule through McMaster-affiliated email verification so eligibility is transparent and uniform for intended users in the pilot community, rather than an unclear manual gate. That choice supports trust between strangers and aligns riders and drivers on the same community boundary, as discussed in Section 4 and the verification-email iteration in Section 3.

For payments, we followed Canadian privacy expectations and PCI practice by keeping card data out of our database and delegating card handling to Stripe, which reduces the security burden on users who are not payment experts. For safety and fairness after a trip, we implemented reporting and related flows (see the Safety & Reporting module in the design documentation) and separated administration into a dedicated web portal so privileged oversight is not a hidden mode inside consumer apps—consistent with Section 3. Together, those measures address power imbalance: riders and drivers get documented channels while operators have dedicated tooling for escalation.

The SRS commits the product to WCAG 2.2 Level AA for the in-app UI, we treated that as a design constraint during implementation and review, while recognizing that a full independent accessibility audit was beyond our capstone scope. We validated workflows with real users, not only internal assumptions with moderated usability sessions (ten participants, task success and timing, and SUS scoring summarized in Section 1 and the usability report under `docs/UsabilityTesting/`) surfaced friction we then addressed in an interface pass. We also shipped role-specific first-use tutorials (Section 2, Extras) so first-time riders and drivers receive in-product guidance instead of inferring every primary action from a cold start, supporting learnability and reducing dependence on prior smartphone-app literacy.

The pilot remains English-first and McMaster-scoped (institutional email and campus-focused use cases, not a general-public launch). Broader language support and opening to a wider population would need additional equity review. Verification depth is lighter than a commercial rideshare platform. For accessibility, we aimed to meet the documented standard in shipped screens but did not commission a third-party WCAG certification, a production roadmap would include deeper assistive-technology testing and continuous checks as the UI evolves.

8 Reflection on Capstone

Burhan:
I personally learned a lot from this capstone project. First off, I entered this project without any experience in working on mobile developement. This project has helped me significantly upskill myself by allowing me to expand my learning in multiple areas and languages, such as TypeScript, React Native, and Expo. I have also learned a lot about time management and effective team work during times of challenges and strict deadlines. Overall, I am very content with the learning experience I had during this project.
Sarim:
Capstone allowed me to use class material we learned in a practical way. I learned how to write an SRS document, design software architecture, and implement a mobile application. I also learned how to work effectively in a team, manage my time, and communicate with my teammates. I am proud of the work we did on this project and the skills I gained from it.
Hamzah:
During the capstone project, I was able to apply the concepts and theories I learned throughout my degree to a real-world project. It was a great learning experience being able to take owenership if a project from start to finish. This includes the documentation, individual features which we developed, and how to effectively collaborate to make our project integrate and work as a whole. I believe I gained valuable experience in software development and project management, especially with mobile development which I had limited experience with prior to this project, and I truly think this will help me in my future career.
Swesan:
Through this capstone, one of the clearest lessons for me was the importance of taking initiative and providing direction. When nobody steps up to propose next steps or clarify priorities, work stalls and the team loses momentum even if everyone intends to contribute.
I also learned how much alignment matters in practice. If we are not on the same page about goals, scope, and deadlines, it is very hard to get anywhere, because effort spreads in different directions instead of compounding on shared outcomes.
Finally, I saw that ownership is a big part of making teamwork work. Being explicit about who owns each task or area keeps responsibilities from staying ambiguous, reduces duplicated effort and dropped work, and makes accountability and follow-through much clearer. Overall, these project-management lessons were as valuable to me as the technical side of the capstone, and I expect to apply them in future team settings.

8.1 Which Courses Were Relevant

This project involved systems design, software architecture, software requirements engineering, software verification and validation, and software development. Here are some of the courses that were relevant to this project:

- Software Requirements Engineering (SFWRENG 3RA3): This course helped us understand requirements engineering and prepare us to write the SRS document.
- Software Design and Architecture (SFWRENG 3A04): This course helped us with understanding software architechture and mobile developement. It also helped us prepare the design documentations for the project.
- Software Verification and Validation (SFWRENG 3S03): This was a comprehensive course on software testing and quality assurance. It helped us prepare the VnV plan and report for our project.
- Databases (SFWRENG 3DB3): This course helped us build a foundation for understanding databases, which was essential in helping us build our database schema and run queries for our project.

8.2 Knowledge/Skills Outside of Courses

This project was heavily docused on mobile development. While the 3A04 course helped us prepare for designing a strong srchitecture, the laugnuges and frameworks we used for the mobile app were not covered in our courses. We had to learn React Native and Expo on our own to be able to build the mobile app for our project. We also had to learn about third-party services such as Stripe for payment processing, Google Maps for location and routing, and Brevo for transactional email, which were not covered in our courses but were essential for implementing key features of our app. Overall, we had to learn a bunch of new knowledge and skills related to mobile development and API integration to build this project.